

Algorithm: Search and Add Node in a Binary Search Tree

Input:

- val → integer value to search/insert
- root → pointer to the root of the BST

Output:

- Updated root of the BST

Steps

1. Check if tree is empty

- If root == NULL:
 - Create a new node with data = val, left = NULL, right = NULL.
 - Set root = newNode.
 - Return root.

2. Initialize traversal pointers

- Set prev_node = root.
- Set curr_node = root.

3. Traverse the tree

- While curr_node != NULL:
 - If val == curr_node->data:
 - Value already exists → stop traversal.
 - Else if (val < curr_node->data)
 - Move curr_node to its left child.
 - Else:
 - Move curr_node to its right child.
 - Update prev_node to the last non-null curr_node.

4. Insert new node if not found

- If value was not found:
 - Create a new node with data = val, left = NULL, right = NULL.
 - If (val < prev_node->data)

- Attach new node as prev_node->left.
- Else:
 - Attach new node as prev_node->right.

5. Return root

- Return the (possibly updated) root pointer.